

<2장 불연산>

목표: 반가산기, 전가산기, 가산기, 증분기, ALU

6) Signed Binary Numbers

이 단원에서 설명하고자 하는 바는 결국,....

어떻게 컴퓨터는 음수를 표현하는가? 이다.

음수를 표현하는 기본적인 방법은 (1)Sign Magnitude(부호 절대값)방법이 있다.

(1)Sign-Magnitude 방법

최상위 비트 MSB 이용. MSB=1이면 그냥 부호만 - 이다.

ex) 1001(2진수) 은 -1(10진수)이다.

장점: 이 방법은 사람이 직관적으로 사용하기에 용이하다.

단점: 양수와 양수의 덧셈이나 음수와 음수의 덧셈은 어렵지 않지만,

음수와 양수의 덧셈은 절대값을 고려해야하기 때문에 회로를 만들기 번거롭다.

또한, +0, -0이 존재한다.

앞서 말했듯이 이러한 수 체계는 "컴퓨터 세상"에서 편의하게 연산을 하기 위함이다.

그런데, 컴퓨터는 가산기만을 이용하여 연산을 하기 때문에, 뺄셈을 덧셈을 변환하여 계산한다.

$A-B = A+(-B)$.이러한 관점에서 보수라는 개념이 등장한다.

보수란?

보수는 보충해주는 수. 즉, 어떤 수를 만들기 위해 필요한 수이다.

ex) 4(10진수)의 10의 보수는 6이다.

ex) 2(10진수)의 10의 보수는 8이다.

보수란 개념을 이용하면 음수를 만드는 다른 방법을 이용할 수 있다. (2) 1의 보수. (3) 2의 보수이다.

2가지 방식 모두 Sign-Magnitude와 달리, n비트의 MSB는 $-2^{(n-1)}$ 을 나타낸다.

(2). 1의 보수

어떤 수 n과의 합이 1이 되는 수이다. ex) 0011의 보수는 1100이다.

왜 이렇게 정의하는지 잘 이해는 안가는데 <합해서 각 자리가 1이 되는 수> 라고 이해하면 좋다.

1의 보수는 양수를 써두고 각 비트를 반전 시켜서 음의 값을 표현한다.

ex) 01(2)은 +1, 00(2)은 +0, 11(2)은 -0, 10(2)은 -1이다.

여기서 보이듯이 +0과 -0이 구분되기 때문에 이것이 단점이다.

(3). 2의 보수

1의 보수를 보완한 방식이다. <01(2)은 +1, 00(2)은 +0, 11(2)은 -0, 10(2)은 -1>인 1의 보수를 보자.

-0이 1의 보수의 문제이므로 -0을 정의 하지 않으려면 양수의 값을 반전하고 +1을 더하면 된다.

그러면 -0은 100(2)이 되어 정의가 불가능해진다.(물리적으로 2비트로 고정되어있다는 가정이기 때문)

위의 설명은 1의 보수를 보완하는 입장의 이해이다.

이런 이해와 달리 그냥 어떻게 쓰는지 이해하는 방식도 있다.

1의 보수에서 합해서 각 자리가 1이 되는 수라고 정의 했으니

2의 보수는 1의 보수에 +1만 하면 되는 것이다.

보수가 결국 어떤 수를 만드는 데 필요한 수라고 했으니,

1을 만드는데 부족한 수를 구하고 +1을 더하면 결국, 2를 만드는 수가 된다. 즉, "2의 보수"

(1을 만드는데 부족한 수 = 1의 보수. 1의 보수는 <합해서 각 자리가 1이 되는 수>)

ex) 0101(2)는 +5(10), 1011(2)는 -5(10)

*Range of N-bit numbers

System	Range
Unsigned	$[0, 2^n-1]$
Sign/Magnitude	$[-2^{(n-1)}+1, 2^{(n-1)}-1]$
Two's Complement	$[-2^{(n-1)}, 2^{(n-1)}-1]$

*32비트 체계에서는 어드레스/데이터/레지스터는 전부 32비트체계로 사용

*자리올림비트(carry bit)/LSB/MSB

*MSB까지 더하고도 CarryBit가 있으며 Overflow

*두 개의 2진수 덧셈시 한 쌍의 비트와 자리올림 비트 합산하는 논리게이트 구성하면 된다.

*프로그램 파일에서 bin파일은 바이터리파일을 의미한다. 이진 텍스트 파일

*Two's complement에서 n비트의 MSB는 $-2^{(n-1)}$ 이다.

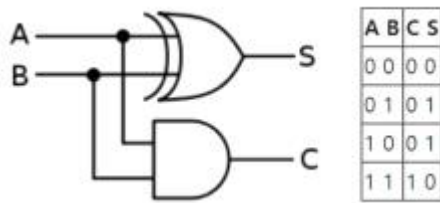
X에서 -X를 구하려면 0을 1로 1을 0으로 바꾸고 1 더한다.

2의 보수는 양수와 음수끼리의 덧셈방식이 같다.

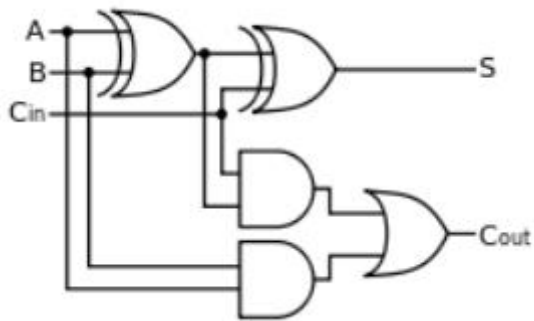
즉, $X-Y=X+(-Y)$ 이다.

ALU에서 덧셈을 하면서 뺄셈을 구현할 수 있다.

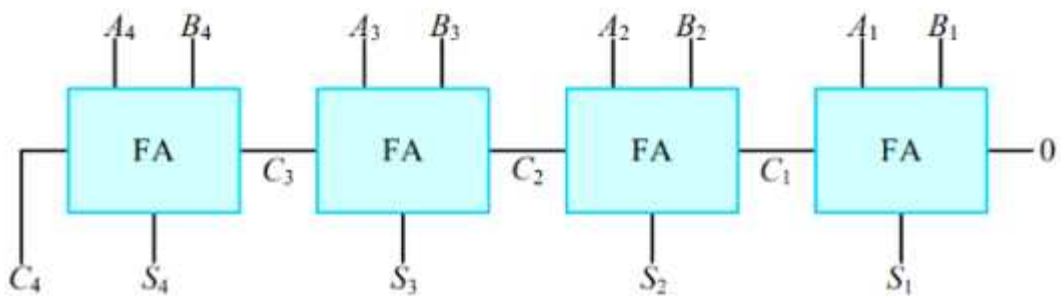
*반가산기 : 2비트를 더함



*전가산기 : 3비트를 더함 : A와 B를 반가산기에 넣고, (A+B)와 Cin을 반가산기에 넣는다.



*가산기 : 두 개의 n비트 숫자 더해

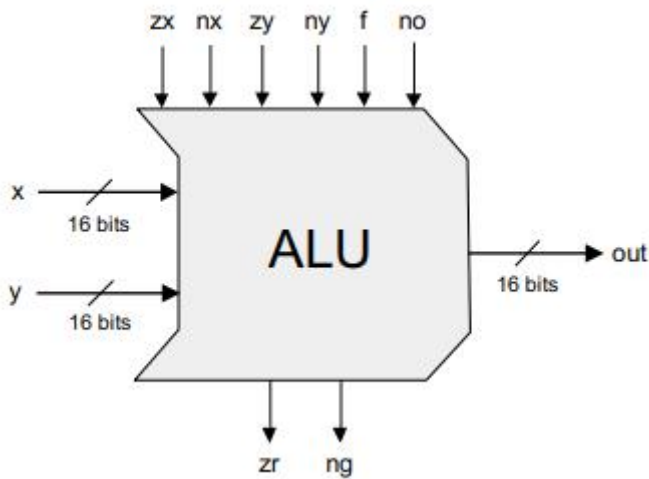


최하위 비트에서 최상위 비트까지 자리올림 비트를 전달하는 과정에서 지연 시간이 길기 때문에 자리올림수 예측기법(Carry look ahead technique)사용한다.

*증분기 : 주어진 숫자에 1 더하는 가산기

[실습 ->ALU, 16비트 가산기, 전가산기,반가산기, 16비트 증분기]

pre-setting the x input		pre-setting the y input		selecting between computing + or &	post-setting the output	Resulting ALU output
zx	nx	zy	ny	f	no	out
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x&y	if no then out=!out	out(x,y)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x y



*zr하고 ng는 다음 장 CPU설계에서 다룬다

1.전가산기

```
CHIP FullAdder {
    IN a, b, c; // 1-bit inputs
    OUT sum, // Right bit of a + b + c
        carry; // Left bit of a + b + c

    PARTS:
    //// Replace this comment with your code.
    Xor(a=a,b=b,out=nodeA);
    Xor(a=nodeA, b=c, out=sum);
    And(a=a,b=b,out=nodeC);
    And(a=nodeA,b=c,out=nodeB);
    Or(a=nodeB, b=nodeC, out=carry);

}
```

2.반가산기

```
CHIP HalfAdder {
    IN a, b; // 1-bit inputs
    OUT sum, // Right bit of a + b
        carry; // Left bit of a + b

    PARTS:
    //// Replace this comment with your code.
    Xor(a=a,b=b,out=sum);
    And(a=a,b=b,out=carry);

}
```

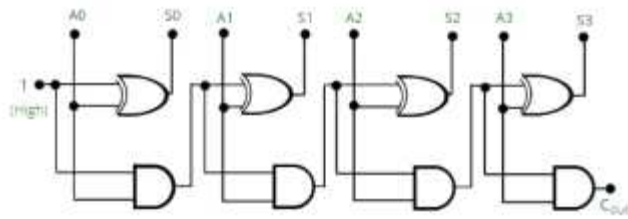
3.16비트 가산기

```
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    //// Replace this comment with your code.
    FullAdder(a=a[0], b=b[0], sum=out[0], carry=n);
    FullAdder(a=a[1], b=b[1], c=n, sum=out[1], carry=n1);
    FullAdder(a=a[2], b=b[2], c=n1, sum=out[2], carry=n2);
    FullAdder(a=a[3], b=b[3], c=n2, sum=out[3], carry=n3);
    FullAdder(a=a[4], b=b[4], c=n3, sum=out[4], carry=n4);
    FullAdder(a=a[5], b=b[5], c=n4, sum=out[5], carry=n5);
    FullAdder(a=a[6], b=b[6], c=n5, sum=out[6], carry=n6);
    FullAdder(a=a[7], b=b[7], c=n6, sum=out[7], carry=n7);
    FullAdder(a=a[8], b=b[8], c=n7, sum=out[8], carry=n8);
    FullAdder(a=a[9], b=b[9], c=n8, sum=out[9], carry=n9);
    FullAdder(a=a[10], b=b[10], c=n9, sum=out[10], carry=n10);
    FullAdder(a=a[11], b=b[11], c=n10, sum=out[11], carry=n11);
    FullAdder(a=a[12], b=b[12], c=n11, sum=out[12], carry=n12);
    FullAdder(a=a[13], b=b[13], c=n12, sum=out[13], carry=n13);
    FullAdder(a=a[14], b=b[14], c=n13, sum=out[14], carry=n14);
    FullAdder(a=a[15], b=b[15], c=n14, sum=out[15], carry=overflowC);

}
```

4.16비트 증분기(Incrementer)



4- Bit Binary Incrementer (In detail)

$$\begin{array}{r}
 \text{Carry: } 0 \ 0 \ 1 \\
 \hline
 \mathbf{1 \ 1 \ 0 \ 1} \\
 \text{(A3) (A2) (A1) (A0)} \\
 + \mathbf{1} \\
 \hline
 \text{Sum: } \mathbf{1 \ 1 \ 1 \ 0}
 \end{array}$$

Working of 4 Bit Binary Incrementer

A0는 초기값, S0는 +1한 후 변환 값.

```

CHIP Inc16 {
    IN in[16];
    OUT out[16];

    PARTS:
        Xor(a=in[0],b=true,out=out[0]);
        And(a=in[0],b=true,out=n1);
        Xor(a=in[1],b=n1,out=out[1]);
        And(a=in[1],b=n1,out=n2);
        Xor(a=in[2],b=n2,out=out[2]);
        And(a=in[2],b=n2,out=n3);
        Xor(a=in[3],b=n3,out=out[3]);
        And(a=in[3],b=n3,out=n4);
        Xor(a=in[4],b=n4,out=out[4]);
        And(a=in[4],b=n4,out=n5);
        Xor(a=in[5],b=n5,out=out[5]);
        And(a=in[5],b=n5,out=n6);
        Xor(a=in[6],b=n6,out=out[6]);
        And(a=in[6],b=n6,out=n7);
        Xor(a=in[7],b=n7,out=out[7]);
        And(a=in[7],b=n7,out=n8);
        Xor(a=in[8],b=n8,out=out[8]);
        And(a=in[8],b=n8,out=n9);
        Xor(a=in[9],b=n9,out=out[9]);
        And(a=in[9],b=n9,out=n10);
        Xor(a=in[10],b=n10,out=out[10]);
        And(a=in[10],b=n10,out=n11);
        Xor(a=in[11],b=n11,out=out[11]);
        And(a=in[11],b=n11,out=n12);
        Xor(a=in[12],b=n12,out=out[12]);
        And(a=in[12],b=n12,out=n13);
        Xor(a=in[13],b=n13,out=out[13]);
        And(a=in[13],b=n13,out=n14);
        Xor(a=in[14],b=n14,out=out[14]);
        And(a=in[14],b=n14,out=n15);
        Xor(a=in[15],b=n15,out=out[15]);
        And(a=in[15],b=n15,out=carryOut);
    }

```

5. ALU

The Hack ALU operation: Compute $y-x$

pre-setting the x input		pre-setting the y input		selecting between computing + or &		post-setting the output	Resulting ALU output
zx	nx	zy	ny	f	no	out	
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x&y	if no then out=!out	out(x,y)=	
1	0	1	0	1	0	0	
1	1	1	1	1	1	1	
1	1	1	0	0	0	-1	
0	0	1	1	0	0	x	
1	1	0	0	0	0	y	
0	0	1	1	0	0	-x	
1	1	0	0	0	0	-y	
0	0	1	1	0	0	x+1	
1	1	0	0	0	0	y+1	
0	0	1	1	0	0	x-1	
1	1	0	0	0	0	y-1	
0	0	1	1	0	0	x+y	
0	0	0	0	0	0	x-y	
0	0	0	0	0	0	y-x	
0	0	0	0	0	0	x&y	
0	1	0	1	0	1	x y	

Example: compute $y-x$

x: 0 0 1 0 (2)

y: 0 1 1 1 (7)

Following pre-setting:

x: 0 0 1 0

y: 1 0 0 0

Compute and post-set:

x+y: 1 0 1 0

!(x+y): 0 1 0 1 (5)

The Hack ALU operation: Compute $!x$

pre-setting the x input		pre-setting the y input		selecting between computing + or &		post-setting the output	Resulting ALU output
zx	nx	zy	ny	f	no	out	
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x&y	if no then out=!out	out(x,y)=	
1	0	1	0	1	0	0	
1	1	1	1	1	1	1	
1	1	1	0	1	0	-1	
0	0	1	1	0	0	x	
1	1	0	0	0	0	y	
0	0	1	1	0	0	-x	
1	1	0	0	0	0	-y	
0	0	1	1	0	0	x+1	
1	1	0	0	0	0	y+1	
0	0	1	1	0	0	x-1	
1	1	0	0	0	0	y-1	
0	0	1	1	0	0	x+y	
0	0	0	0	0	0	x-y	
0	0	0	0	0	0	y-x	
0	0	0	0	0	0	x&y	
0	1	0	1	0	1	x y	

Example: compute $!x$

x: 1 1 0 0

y: 1 0 1 1 (don't care)

Following pre-setting:

x: 1 1 0 0

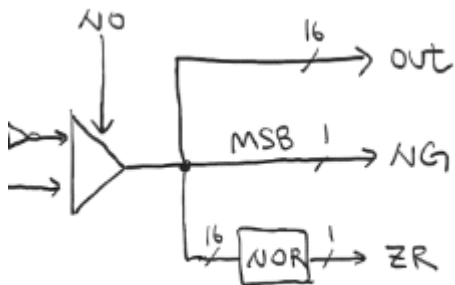
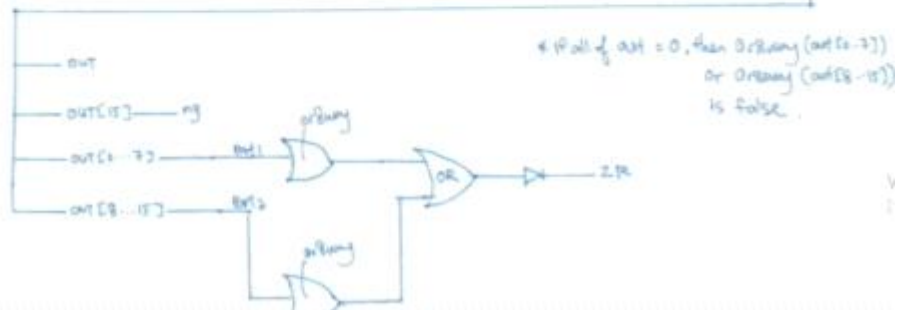
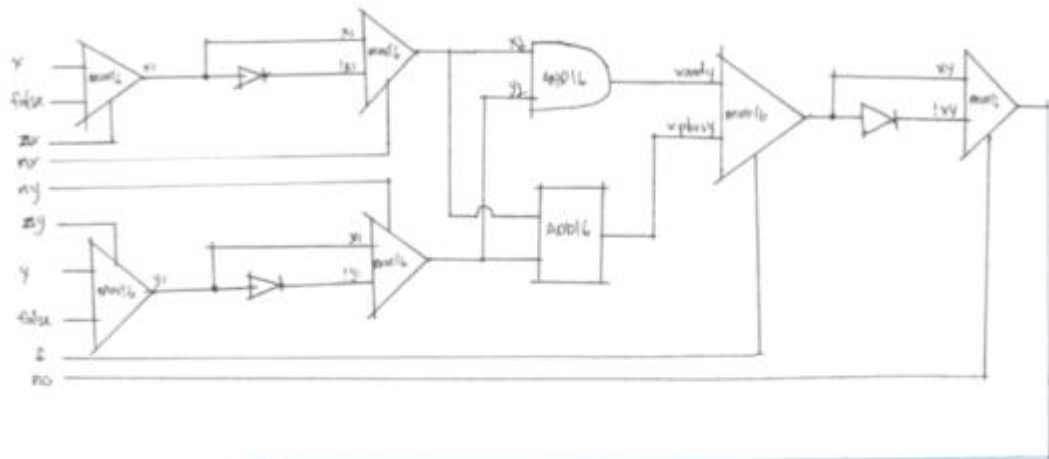
y: 1 1 1 1

Compute and post-set:

x&y: 1 1 0 0

!(x&y): 0 0 1 1 (!x)

ALL INDEPENDENT



ZR은 NOR 하나면 되고 out과 false 걸면된다.

위의 Or8Way 2개와 Or로 Or8Way만든 것이고 Not을 써서 Nor 구현 한 것이다.
1이 하나도 없을 때(0일 때) 1이 나오는 논리회로.

```

// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/ALU.hdl
/**
 * ALU (Arithmetic Logic Unit):
 * Computes out = one of the following functions:
 *
 *      0, 1, -1,
 *      x, y, !x, !y, -x, -y,
 *      x + 1, y + 1, x - 1, y - 1,
 *      x + y, x - y, y - x,
 *      x & y, x | y
 * on the 16-bit inputs x, y,
 * according to the input bits zx, nx, zy, ny, f, no.
 * In addition, computes the two output bits:
 * if (out == 0) zr = 1, else zr = 0
 * if (out < 0)  ng = 1, else ng = 0
 */
// Implementation: Manipulates the x and y inputs
// and operates on the resulting values, as follows:
// if (zx == 1) sets x = 0      // 16-bit constant
// if (nx == 1) sets x = !x    // bitwise not
// if (zy == 1) sets y = 0      // 16-bit constant
// if (ny == 1) sets y = !y    // bitwise not
// if (f == 1)  sets out = x + y // integer 2's complement addition
// if (f == 0)  sets out = x & y // bitwise and
// if (no == 1) sets out = !out  // bitwise not

CHIP ALU {
    IN
        x[16], y[16], // 16-bit inputs
        zx, // zero the x input?
        nx, // negate the x input?
        zy, // zero the y input?
        ny, // negate the y input?
        f, // compute (out = x + y) or (out = x & y)?
        no; // negate the out output?
    OUT
        out[16], // 16-bit output
        zr,      // if (out == 0) equals 1, else 0
        ng;      // if (out < 0) equals 1, else 0
}

```

PARTS:

//// Replace this comment with your code.

//x presetting

Mux16(a=x,b=false,sel=zx, out=zxNode);

Not16(in=zxNode, out=ngxNode);

Mux16(a=zxNode,b=ngxNode,sel=nx, out=nxNode);

//y presetting

Mux16(a=y,b=false,sel=zy, out=zyNode);

Not16(in=zyNode, out=ngyNode);

Mux16(a=zyNode,b=ngyNode,sel=ny, out=nyNode);

//function

And16(a=nxNode,b=nyNode,out=andNode);

Or16(a=nxNode,b=nyNode,out=orNode);

Mux16(a=andNode,b=orNode,sel=f,out=funcNode);

//revVal

Not16(in=funcNode, out=ngRev);

Mux16(a=funcNode,

b=ngRev, sel=no,out=out,out[0..7]=firstNode,out[8..15]=secondNode,out[15]=msb);

//zr (out==0) Use Nor

Or8Way(in=firstNode,out=zr1);

Or8Way(in=secondNode,out=zr2);

Or(a=zr1, b=zr2, out=zrNode);

Not(in=zrNode,out=zr);

//ng (out<0) Use MSB out[15]

And(a=true, b=msb,out=ng);

}